

Tech Stack & Architecture

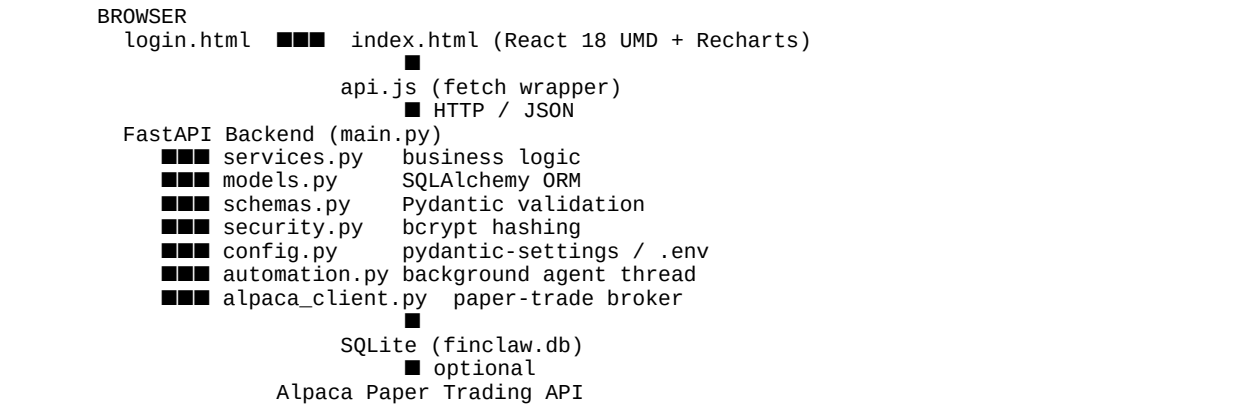
FinClaw System Design Reference

FinClaw — AI-Assisted Paper Trading Dashboard

1. Project Overview

FinClaw is a lightweight AI-assisted paper-trading dashboard. Users onboard, set risk policies, view portfolio summaries, chat with an AI assistant, and submit guarded trade intents — all backed by a FastAPI service that optionally connects to Alpaca's paper-trading API.

2. High-Level Architecture



3. Layer Breakdown

Layer	Technology	Role
UI	HTML + Tailwind CDN + React 18 UMD + Recharts	Dashboard, forms, charts
API Client	api.js (fetch)	Browser ↔ Backend bridge
Backend	FastAPI + Uvicorn	REST API, routing, validation
ORM	SQLAlchemy 2	Database access layer
Database	SQLite	Persistent local storage
Validation	Pydantic v2	Request/response schemas
Auth	passlib bcrypt	Password hashing
Broker	Alpaca SDK (optional)	Paper-trade execution
Agent	Python threading	Autonomous trading loop
Container	Docker + docker-compose	Local deployment

4. Backend Modules

File	Responsibility
main.py	FastAPI app, all routes, CORS, global error handler, lifespan
services.py	Auth, policies, portfolio, chat, trade validation, audit
models.py	SQLAlchemy ORM: User, Policy, Trade, AuditLog

schemas.py	Pydantic request/response models with field-level validation
config.py	Settings class, .env loading, approved tickers list
security.py	hash_password / verify_password via passlib bcrypt
database.py	Engine, SessionLocal, Base, get_db dependency
automation.py	TradingAutomationAgent — threaded loop, state management
alpaca_client.py	Real Alpaca SDK call or simulated fallback
logging_config.py	Structured JSON logging setup

5. Data Models

Model	Key Fields
User	id, email, password, goal, risk, experience, asset, created_at
Policy	id, user_email, title, value, enabled
Trade	id, user_email, asset, trade_type, amount, quantity, pnl_percent, status, auto_ai, execution_reason, created_at
AuditLog	id, user_email, action, decision, reason, payload, created_at

6. Tech Stack Summary

Category	Choice	Notes
Language	Python 3.11+	
Web Framework	FastAPI	Auto OpenAPI docs
ASGI Server	Uvicorn	--reload for dev
ORM	SQLAlchemy 2	mapped_column style
Validation	Pydantic v2	field_validator support
Settings	pydantic-settings	.env file loading
Auth	passlib[bcrypt]	hash + verify
Database	SQLite	finclaw.db
Broker SDK	alpaca-trade-api	Optional; simulated fallback
Frontend	React 18 UMD	No build step, CDN
Styling	Tailwind CSS CDN	JIT via CDN
Charts	Recharts 2 UMD	AreaChart, PieChart
JSX	Babel Standalone	In-browser transpile
Container	Docker + Compose	Multi-service local stack
Testing	pytest + TestClient	backend/tests/test_api.py